

Project 1: Convolution

Part 1 – High-Level Implementation (Java)

```
/**
 * Demonstrating 1-D convolution on an array of integers
 */
public class Convolution {
    /**
     * Performs simple convolution on integer array.
     * New array replaced with average of itself and neighbors.
     *
     * @param A input array of int to use
     * @return new array containing the average values
     */
    public static int[] convolution(int[] A) {

        int N = A.length;
        int[] B = new int[N];

        for (int i = 0; i < N; i++) {
            int sum = A[i];

            if (i > 0) {
                sum += A[i - 1]; // adding left neighbor
            }

            if (i < N - 1) {
                sum += A[i + 1]; // adding right neighbor
            }

            B[i] = sum / 3; // average to be added to B
        }

        return B;
    }

    public static void printArray(int[] arr) {
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }

        System.out.println();
    }
}
```

```
public static void main(String[] arts) {  
  
    // Test_1  
    int[] A1 = { 9, 20, 21, 8, 11, 12, 3, 15 };  
    int[] B1 = convolution(A1);  
  
    System.out.println("Test_1");  
    System.out.print("Array A: ");  
    printArray(A1);  
  
    System.out.print("Array B: ");  
    printArray(B1);  
    System.out.println();  
  
    // Test_2  
    int[] A2 = { 1, 1, 1, 1, 1 };  
    int[] B2 = convolution(A2);  
  
    System.out.println("Test_2");  
    System.out.print("Array A: ");  
    printArray(A2);  
  
    System.out.print("Array B: ");  
    printArray(B2);  
    System.out.println();  
  
    // Test_3  
    int[] A3 = { 100, 50, 25, 130, 200, 10 };  
    int[] B3 = convolution(A3);  
  
    System.out.println("Test_3");  
    System.out.print("Array A: ");  
    printArray(A3);  
  
    System.out.print("Array B: ");  
    printArray(B3);  
  
    }  
}
```

Part 2 – RISC-V Assembly Implementation

.data

arrA: .string "Array A: "

arrB: .string "\nArray B: "

space: .string " "

A: .word 100, 50, 25, 130, 200, 10 # Change for testing

Output change for testing

B: .space 24 # 5 int * 4 bytes

N: .word 6 # array length

.text

.globl main

main:

la s0, A

la s1, B

lw s2, N

li t0, 0 # Initialize loop i starting at 0

loop:

bge t0, s2, end_loop # if i >= N end

slli t1, t0, 2 # byte offset for array access

add t2, s0, t1 # Add offset to A = A[i]

lw t3, 0(t2)

beq t0, x0, skip_left # if i == 0 skip, left neighbor check

addi t4, t0, -1

slli t4, t4, 2 # offset

add t5, s0, t4 # Memory address of A[i - 1]

```
lw t6, 0(t5)
```

```
add t3, t3, t6      # Add left neighbor
```

```
skip_left:
```

```
addi t4, s2, -1     # N - 1 check for last input
```

```
beq t0, t4, skip_right  # if i == N - 1 skip, right neighbor check
```

```
addi t4, t0, 1      # index i + 1
```

```
slli t4, t4, 2      # offset
```

```
add t5, s0, t4      # Memory address A[i + 1]
```

```
lw t6, 0(t5)
```

```
add t3, t3, t6      # Add right neighbor
```

skip_right:

li t4, 3

div t3, t3, t4

add t5, s1, t1 # Memory address B[i]

sw t3, 0(t5) # Store into arr B

addi t0, t0, 1 # increment index looping

j loop

end_loop:

la a0, arrA # Array A for print

li a7, 4 # System call print string

ecall

li t0, 0 # Reset counter print Array A

printA:

bge t0, s2, printB # move to Array B

slli t1, t0, 2 # offset A[i]

add t2, s0, t1

lw a0, 0(t2)

li a7, 1 # System call print int

ecall

la a0, space

li a7, 4

ecall

addi t0, t0, 1

```
j printA
```

```
printB:
```

```
la a0, arrB
```

```
li a7, 4
```

```
ecall
```

```
li t0, 0
```

```
printBloop:
```

```
bge t0, s2, exit    # if all print exit
```

```
slli t1, t0, 2
```

```
add t2, s1, t1
```

```
lw a0, 0(t2)
```


li a7, 1

ecall

la a0, space

li a7, 4

ecall

addi t0, t0, 1

j printBloop

exit:

li a7, 10

ecall

Part 3 – Write Up

The algorithm for convolution project is the input array's elements and its neighbors being averaged. Each new output is then added to the new array B by taking the current index of array A and adding the current element with its neighboring elements if it exists. The total sum will then be divided by three and produce an average. Then certain conditions must be checked for the first and last elements as they will not have both neighbors. The conditional checks will be making sure the indices checked do not try to perform out of range. 'beq t0, x0, skip_left' t0 being [i] to make sure it does not access an index like A[-1] for the first index. As for the last element check there is total N length and [i] current index if $i == N - 1$ then the process will stop. 'addi t4, s2 -1' and 'beq t0, t4, skip_right'

The high-level implementation was simpler. Java being used has auto management of array indexing and arithmetic. These processes made it less troublesome dealing with the RISC-V Assembly of managing the memory addresses. As in accessing A[i], A[i - 1], and A[i + 1]. Translating the algorithm to RISC-V required computing every step and off-set. Manual calculations of these steps can come with errors as there are more steps involved. A challenge at first was figuring out the first and last elements looping requirements to skip calculations if it did not have a second neighbor.

Screenshots of Tests

*First screenshot has typo in the last test should be Test_3.

The image shows a screenshot of an IDE (likely IntelliJ IDEA) with a Java file named `Convolution.java` open. The code is a simple 1D convolution implementation. Below the code editor, the `TERMINAL` tab is active, showing the output of the program. The output displays two test cases, `Test_1` and `Test_2`, each with input arrays `A` and `B`.

```
1 /**
2  * Demonstrating 1-D convolution on an array of integers
3  */
4 public class Convolution {
5
6     /**
7      * Performs simple convolution on integer array.
8      * New array replaced with average of itself and neighbors.
9      *
10     * @param A input array of int to use
11     * @return new array containing the average values
12     */
13     public static int[] convolution(int[] A) {
14
15         int N = A.length;
16         int[] B = new int[N];
17
18         for (int i = 0; i < N; i++) {
19             int sum = A[i];
20
21             if (i > 0) {
22                 sum += A[i - 1]; // adding left neighbor
23             }
24
25             if (i < N - 1) {
26                 sum += A[i + 1]; // adding right neighbor
27             }
28
29             B[i] = sum / 3;
30         }
31
32         return B;
33     }
34 }
```

The terminal output shows the following results:

```
To update your account to use zsh, please run 'chsh -s /bin/zsh'.
For more details, please visit https://support.apple.com/kb/HT208050.
Mac:~> sabers /usr/bin/env /Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java --enable-preview -XX:+ShowCodeDetailsInExceptionMessages -cp /private/var/folders/v2/xfn79309Shdkn
r_88ghrp888888n/T/vscodesws_a08bd/jdt_ws/jdt.ls-java-project/bin Convolution
Test_1
Array A: 9 20 21 8 11 12 3 15
Array B: 9 16 16 13 10 8 10 6

Test_2
Array A: 1 1 1 1 1
Array B: 0 1 1 1 0

Test_2
Array A: 100 50 25 130 200 10
Array B: 50 50 68 118 113 70
Mac:~> sabers
```

Below the terminal output, there is a section for the assembly code (P1.asm) and a table of registers. The assembly code is shown in the `Text Segment` and `Data Segment` tabs. The registers table shows the state of various registers, including `zero`, `ra`, `sp`, `gp`, `tp`, `t0`, `t1`, `t2`, `s0`, `s1`, `a0`, `a1`, `a2`, `a3`, `a4`, `a5`, `a6`, `a7`, `s2`, `s3`, `s4`, `s5`, `s6`, `s7`, `s8`, `s9`, `s10`, `s11`, `t3`, `t4`, `t5`, `t6`, and `pc`.

Name	Number	Value
zero	0	0
ra	1	0
sp	2	2147479548
gp	3	268468224
tp	4	0
t0	5	0
t1	6	28
t2	7	268501076
s0	8	268501016
s1	9	268501040
a0	10	268501013
a1	11	0
a2	12	0
a3	13	0
a4	14	0
a5	15	0
a6	16	0
a7	17	10
s2	18	8
s3	19	0
s4	20	0
s5	21	0
s6	22	0
s7	23	0
s8	24	0
s9	25	0
s10	26	0
s11	27	0
t3	28	6
t4	29	3
t5	30	268501076
t6	31	3
pc		4194568

The assembly code is shown in the `Text Segment` and `Data Segment` tabs. The `Text Segment` shows the instructions for the convolution function, and the `Data Segment` shows the input arrays `A` and `B`.

Below the assembly code, there is a section for the messages and I/O. The messages section shows the output of the program, and the I/O section shows the input and output of the program.

Messages: Run I/O

Array A: 9 20 21 8 11 12 3 15
Array B: 9 16 16 13 10 8 10 6
-- program is finished running (0) --

Clear

File Edit Run Settings Tools Help

Run speed at max (no interaction)

Edit Execute

Text Segment

Bkpt	Address	Code	Basic	Source
	4194304	0x0fc10417 auipc x8,64528		26: 1a s0, A
	4194308	0x01840413 addi x8,x8,24		
	4194312	0x0fc10497 auipc x9,64528		28: 1a s1, B
	4194316	0x02484893 addi x9,x9,40		

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+12)	Value (+16)	Value (+20)	Value (+24)	Value (+28)
268500992	1634890305	977346681	1091174432	2036429426	540688928	8192	100	50
268501024	25	130	200	10	50	58	68	118

0x10010000 (.data) Hexadecimal Addresses Hexadecimal Values ASCII

Messages Run I/O

Clear

Array A: 9 20 21 8 11 12 3 15
Array B: 9 16 16 13 10 8 10 6
-- program is finished running (0) --
Array A: 1 1 1 1 1
Array B: 0 1 1 1 0
-- program is finished running (0) --
Array A: 100 50 25 130 200 10
Array B: 50 58 68 118 113 70

Registers Floating Point Control and Status

Name	Number	Value
zero	0	0
ra	1	0
sp	2	2147479548
gp	3	268468224
tp	4	0
t0	5	0
t1	6	20
t2	7	268501060
s0	8	268501016
s1	9	268501040
a0	10	268501013
a1	11	0
a2	12	0
a3	13	0
a4	14	0
a5	15	0
a6	16	0
a7	17	10
s2	18	6
s3	19	0
s4	20	0
s5	21	0
s6	22	0
s7	23	0
s8	24	0
s9	25	0
s10	26	0
s11	27	0
t3	28	70
t4	29	3
t5	30	268501060
t6	31	200
pc		4194568

File Edit Run Settings Tools Help

Run speed at max (no interaction)

Edit Execute

Text Segment

Bkpt	Address	Code	Basic	Source
	4194304	0x0fc10417 auipc x8,64528		26: 1a s0, A
	4194308	0x01840413 addi x8,x8,24		
	4194312	0x0fc10497 auipc x9,64528		28: 1a s1, B
	4194316	0x02484893 addi x9,x9,36		

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+12)	Value (+16)	Value (+20)	Value (+24)	Value (+28)
268500992	1634890305	977346681	1091174432	2036429426	540688928	8192	1	1
268501024	1	1	1	0	1	1	1	0

0x10010000 (.data) Hexadecimal Addresses Hexadecimal Values ASCII

Messages Run I/O

Clear

Array A: 9 20 21 8 11 12 3 15
Array B: 9 16 16 13 10 8 10 6
-- program is finished running (0) --
Array A: 1 1 1 1 1
Array B: 0 1 1 1 0
-- program is finished running (0) --

Registers Floating Point Control and Status

Name	Number	Value
zero	0	0
ra	1	0
sp	2	2147479548
gp	3	268468224
tp	4	0
t0	5	5
t1	6	16
t2	7	268501052
s0	8	268501016
s1	9	268501036
a0	10	268501013
a1	11	0
a2	12	0
a3	13	0
a4	14	0
a5	15	0
a6	16	0
a7	17	10
s2	18	5
s3	19	0
s4	20	0
s5	21	0
s6	22	0
s7	23	0
s8	24	0
s9	25	0
s10	26	0
s11	27	0
t3	28	0
t4	29	3
t5	30	268501052
t6	31	1
pc		4194568